The National School of Artificial Intelligence (ENSIA)

4th Year AI Engineering

High-Performance Computing (HPC)

OpenMPI Lab Report

Lab Sheet 3

Student:
BOUHOUIA Yousra
Group: G1

April 15, 2026

Introduction

In this lab, we worked with MPI in an HPC environment to better understand how parallel processes communicate and cooperate. We executed different programs on the cluster using SLURM scripts and observed how multiple processes interact during execution.

Through this lab, we learned how to divide processes into groups, perform collective operations, and verify results manually. This helped us better understand how communication patterns are used in real distributed systems, especially in AI training where synchronization between processes is important.

We also became more familiar with compiling MPI programs, submitting jobs, and analyzing outputs generated by parallel execution.

Exercise 1: Communicator Splitting & Group Allreduce

In this exercise, we implemented a program where 6 MPI processes are divided into smaller groups of 2 processes each using `MPI_Comm_split`.

Each process calculated a local value using the formula:

```
value = (rank + 1) × 10
```

Then, we used `MPI_Allreduce` inside each group to compute the sum of values for that group. Each process printed its global rank, group ID, local rank, value, and the computed group sum.

What we understood

This exercise helped us understand how MPI allows splitting a global communicator into smaller sub-communicators. Instead of all processes communicating together, each group can work independently.

We also understood that `MPI_Allreduce` performs a collective operation where all processes in the group receive the same result. This is very useful in parallel computing when processes need to synchronize their data.

Another important point is how the grouping is done using the color value. Processes with the same color belong to the same group, which makes the division simple and efficient.

Results

The results obtained matched the expected values:

- Group 0 (ranks 0 and 1): sum = 30
- Group 1 (ranks 2 and 3): sum = 70
- Group 2 (ranks 4 and 5): sum = 110

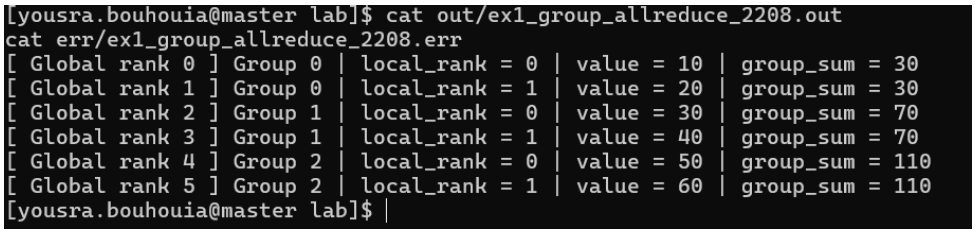

Figure 1: Execution output of Exercise 1

Execution Output

Conclusion

From this exercise, we learned how to organize processes into groups and perform collective operations within those groups. This concept is important in many HPC and AI applications where computations are distributed but still require partial synchronization.

Exercise 2: 2D Cartesian Topology & Neighbor Exchange

In this exercise, we created a 2D Cartesian topology using 6 MPI processes arranged in a grid of 2 rows and 3 columns. We used a periodic topology, which means that the processes at the borders are connected again to the processes on the opposite side. This allowed us to simulate neighbor communication in all four directions: up, down, left, and right.

Each process was assigned a value based on its Cartesian rank, then we exchanged data with the four neighbors. This helped us understand how MPI can organize processes in a structured topology and how each process can identify its coordinates and its surrounding neighbors.

What we understood

Through this exercise, we understood that Cartesian topologies are very useful in parallel computing when the problem has a grid structure, such as simulations or image processing. Using `MPI_Cart_create`, `MPI_Cart_coords`, and `MPI_Cart_shift`, we can easily manage communication between neighboring processes.

We also noticed an important issue related to deadlock. When using blocking communication functions such as `MPI_Send` and `MPI_Recv`, a process may remain blocked while waiting to receive data, while the other process is also waiting. In this case, both processes wait for each other, which causes a deadlock.

We understood that this problem can be solved by using communication methods that do not block in this way. For example, we can use non-blocking operations such as `MPI_Isend` and `MPI_Irecv`, where processes do not wait immediately and can continue execution.

Results

The execution output shows the value of each process and the values received from its neighbors in the Cartesian grid.

```
[yousra.bouhouia@master lab]$ cat out/ex2_cart_exchange_2225.out
cat err/ex2_cart_exchange_2225.err
Rank 0 at (0,0) | my_val=0 | from_up=30 | from_down=30 | from_left=20 | from_right=10
Rank 1 at (0,1) | my_val=10 | from_up=40 | from_down=40 | from_left=0 | from_right=20
Rank 2 at (0,2) | my_val=20 | from_up=50 | from_down=50 | from_left=10 | from_right=0
Rank 3 at (1,0) | my_val=30 | from_up=0 | from_down=0 | from_left=50 | from_right=40
Rank 4 at (1,1) | my_val=40 | from_up=10 | from_down=10 | from_left=30 | from_right=50
Rank 5 at (1,2) | my_val=50 | from_up=20 | from_down=20 | from_left=40 | from_right=30
[yousra.bouhouia@master lab]$
```

Figure 2: Execution output of Exercise 2

Conclusion

From this exercise, we learned how to use Cartesian topologies in MPI and how neighbor exchange works in a 2D grid. We also understood how deadlocks can occur and how to avoid them using appropriate communication methods.

Exercise 3: Flat vs. Hierarchical All-Reduce

In this exercise, we implemented two versions of the all-reduce operation: a flat all-reduce using `MPI_Allreduce` on all processes, and a hierarchical version where processes are divided into groups (simulating nodes).

The hierarchical approach was done in three steps: first, a local reduction inside each group, then an all-reduce between group leaders, and finally a broadcast inside each group.

What we understood

This exercise helped us understand how hierarchical communication is used in large-scale distributed systems. Instead of involving all processes in a single global communication, the work is divided into local and global steps to reduce communication cost.

We also learned how to use `MPI_Comm_split` to simulate nodes and organize processes into leaders and workers.

Results

```
[yousra.bouhouia@master lab]$ cat out/ex3_hierarchical_allreduce_2240.out
cat err/ex3_hierarchical_allreduce_2240.err
Flat Allreduce     : result = 21.0 | time = 0.010903 s
Hierarchical       : result = 21.0 | time = 0.012975 s
Avg Flat latency   : 1.090 us/iter
Avg Hier latency   : 1.298 us/iter
[yousra.bouhouia@master lab]$
```

Figure 3: Execution output of Exercise 3

From the results, we notice that the flat all-reduce is faster than the hierarchical version:

- Flat Allreduce: lower execution time and latency
- Hierarchical Allreduce: slightly higher execution time

At first, this result seems illogical because the hierarchical method is supposed to be more efficient by reducing the number of communications between processes.

Explanation

This result can be explained by the fact that we are working with a very small number of processes (only 6) and within a single node environment. In this case, the communication cost is already low, so adding extra steps in the hierarchical approach (local reduce, leader communication, then broadcast) actually introduces more overhead.

In other words, the hierarchical method adds more operations than necessary for such a small system, which makes it slower instead of faster.

Hierarchical all-reduce becomes efficient only in large-scale systems with many nodes, where reducing inter-node communication significantly improves performance. In our case, since all processes are close (same machine or small cluster), the flat method remains more efficient.

Conclusion

From this exercise, we understood that optimization techniques like hierarchical communication are not always beneficial. Their efficiency depends on the scale of the system. For small numbers of processes, a simple flat all-reduce is often faster, while hierarchical methods are more suitable for large distributed systems.

Exercise 4: Ring Topology – Token Accumulation

In this exercise, we implemented a ring communication pattern using MPI. Each process was arranged in a logical ring, where it sends its token to the next process and receives a token from the previous one. The initial token of each process was calculated using the formula:

```
token = (rank + 1) × 10
```

Then, each process kept a running sum initialized with its own token. During each iteration, the token was passed around the ring, and the received value was added to the local sum. After a full rotation, all processes were expected to obtain the same global sum.

What we understood

Through this exercise, we understood how data circulates in a ring topology and how each process contributes to the final global result. This communication pattern is important in parallel computing because it is the basis of algorithms such as ring-allreduce used in distributed deep learning.

We also understood that each process only communicates with two neighbors: the previous process and the next process. Even with this simple communication pattern, all processes are still able to obtain the complete global sum after enough steps.

Another important point in this exercise is deadlock avoidance. When blocking functions such as `MPI_Send` and `MPI_Recv` are used, the communication order must be carefully managed. This can be done either with even/odd ordering or with `MPI_Sendrecv`, which is simpler and safer.

Results

The execution output shows that all processes obtained the same final sum:

- Rank 0: sum = 100
- Rank 1: sum = 100
- Rank 2: sum = 100
- Rank 3: sum = 100

This matches the expected global sum:

$$E = 10 \times P \times (P + 1) / 2 = 10 \times 4 \times 5 / 2 = 100$$

```
[yousra.bouhouia@master lab]$ cat out/ex4_ring_accumulate_2241.out
cat err/ex4_ring_accumulate_2241.err
Rank 0 : sum = 100 ( expected 100 )
Rank 1 : sum = 100 ( expected 100 )
Rank 2 : sum = 100 ( expected 100 )
Rank 3 : sum = 100 ( expected 100 )
[yousra.bouhouia@master lab]$
```

Figure 4: Execution output of Exercise 4

Conclusion

From this exercise, we learned how ring communication works and how a global sum can be obtained progressively by circulating tokens between neighboring processes. We also understood that this topology is efficient and widely used in distributed systems, especially in collective communication algorithms.